MyCodeSchool

Code review — Cafenea Spring

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-22 — sha 57e0657

Elev: Alex. Revizuite: `f424fb5`, `b183650`, `378ee6c`, `ef1ba58`, `af76641`, `57e0657`. Proiect: `cafenea-spring/`.

Poarta de compilare: `mvn -q -DskipTests compile` trece. Aplicația pornește și rulează.

Ce e bine: nu mai există niciun `getINSTANCE()`; toate dependențele vin prin constructor; `*Impl` au `@Service` / `@Repository`; `Scanner` și `CalculatorPret` sunt bean-uri în `AppConfig`; meniul pornește dintr-un `CommandLineRunner`. Structura etapelor 1–2 e corectă.

Verificat prin rulare (pe o copie, nu în repo-ul tău):

Verificare	Rezultat
Scenariul `2,2,2,4,3,4,5,0`	pică cu `NoSuchElementException` la `ViewCafenea.java:38`
Același scenariu adaptat la meniul tău (`2,2,4,0,3,4,5,0`)	`TOTAL 26.62 lei`, corect
`--cafenea.tva=0.19`	`TOTAL 26.62 lei` (trebuia `26.18`)
Ieșirea față de vanilla	diferă (vezi B1)

🔴 Critice

B1 🔴 Meniul nu mai e cel din vanilla

`ViewCafenea.java:46-67`, `:69-73`, `:85-94`

CUM E ACUM: cerința spune că `2,2,2,4,3,4,5,0` dă *exact aceeași ieșire ca în vanilla*. La tine aplicația se blochează: opțiunea 2 a devenit „Comanda noua" cu o buclă care cere id-uri până la `0`, deci scenariul se termină în mijlocul buclei și `Scanner` n-are ce citi. Diferențe găsite prin rulare:

- Opțiunea 2 creează o comandă nouă de fiecare dată (`:47`). `2,2,0, 2,4,0, 3` arată un bon doar cu Croissant: primul Cappuccino a dispărut fără niciun mesaj.
- Opțiunea 4 pe comandă goală o finalizează (`5` arată `#1 0 produse 0.00 lei`); vanilla răspunde „Comanda e goala".

- După `4`, `comandaCurenta = null` (`:93`), deci `3` zice „Nu exista o comanda activa”; în vanilla se deschide singură o comandă nouă.
- Textele meniului și formatul bonului (`%-15s`, `Subtotal %6.2f`) diferă de vanilla.

DE CE: o migrare la Spring schimbă *cine creează obiectele*, nu *ce face programul*. De aceea verificarea din cerință e un diff de ieșire: dacă scenariul identic dă text identic, știi că ai mutat doar legăturile (constructori, bean-uri) și nimic din logică. Aici ai rescris `ViewCafenea` în loc să o copiezi și să-i scoți doar `getINSTANCE()` și `new Scanner`. Fiecare diferență de comportament e acum un bug față de aplicația de referință, iar testul tău manual (`2,2,4,0,...`) a fost scris pentru meniul tău, nu pentru cerință, deci nu le-ar fi prins.

💡 **Hint:** pornește de la `cafenea-vanilla-java/src/app/view/ViewCafenea.java`. Din el schimbi doar ce cere etapa 1: dependențele vin din constructor și metoda `start()` e apelată de `CommandLineRunner`. Apoi pui aceleași opt rânduri în ambele aplicații și faci `diff` pe ieșiri.

B2🔴 `@PropertySource("classpath:application.yml")` nu citește YAML

`AppConfig.java:12` (commit `57e0657`)

CUM E ACUM: aplicația „merge”, dar adnotarea nu face nimic. Am verificat:

Situație	`cafenea.nume`	`cafenea.program`
`@PropertySource` pe `program.properties` + Boot citește `application.yml`	`Cafenea`	`08:00-20:00`
același, dar fără auto-încărcarea Boot	`LIPSA`	`08:00-20:00`
`@PropertySource` pe `application.yml`, fără Boot	`LIPSA`	`LIPSA`
`@PropertySource` pe `application.yml`, cu Boot (cum rulezi tu)	`Cafenea`	`08:00-20:00`

DE CE: `@PropertySource` știe să citească doar formatul `.properties` (linii `cheie=valoare`). Pe un `.yml` îl parsează tot ca `.properties`: o linie ca `nume: Cafenea` devine cheia `nume` cu valoarea `Cafenea` (și `:` e separator în `.properties`), iar `cafenea:` devine cheia `cafenea` cu valoare goală. Nicio cheie nu are forma `cafenea.nume`, deci nu poate găsi nimic. Ce vezi tu în rulare vine din altă parte: Spring Boot încarcă singur `application.yml` cu un încărcător care înțelege YAML și aplatizează indentarea în `cafenea.nume`. Adnotarea ta e degeaba, iar rularea reușită o maschează: ai crede că a lucrat ea. Rândul 4 din tabel e identic cu rândul 1 tocmai pentru că Boot face toată treaba.

💡 **Hint:** cerința are două fișiere cu roluri diferite. `application.yml` (Boot îl încarcă singur) ține `nume`, `moneda`, `tva`. `program.properties` din `resources/` ține `cafenea.program=08:00-20:00` și este singurul pentru care ai

nevoie de `@PropertySource`, pe `AppConfig`. Rândul 2 din tabel e testul tău: dacă `program` apare chiar și fără Boot, adnotarea lucrează.

B3 🔴 Etapa 3 e făcută pe jumătate

Ai făcut prima jumătate a etapei (fișierul `application.yml`); a doua jumătate, cea care *folosește* valorile din el, lipsește.

Cerință	Starea acum
`CalculatorPret` primește TVA cu `@Value`, devine `@Component`	`AppConfig.java:20–23` încă `new CalculatorPret(0.21)`; `CalculatorPret.java:10` are TVA-ul ca parametru fără `@Value`
În `AppConfig` rămâne un singur `@Bean`: `Scanner`	rămân trei: `scanner`, `calculatorPret`, `pornesteMeniul`
`record CafeneaProperties(nume, moneda, program)` + `@ConfigurationProperties(prefix = "cafenea")` + `@EnableConfigurationProperties`	nu există
Antet `=== Cafenea · 08:00–20:00 ===`	`ViewCafenea.java:29` încă `========== CAFENEA ==========`
`lei` din `moneda`	hardcodat la `ViewCafenea.java:82`; în `:98` lipsește
`--cafenea.tva=0.19` dă `26.18`	dă `26.62`: TVA-ul e scris în cod, `application.yml` nu e citit de nimeni

DE CE contează: acum `tva: 0.21` din yml există dar nimeni nu-l citește; schimbi valoarea în fișier și bonul rămâne neschimbat. Asta e exact diferența dintre „config în fișier" și „config în cod cu un fișier alături". Lanțul complet e: yml → `Environment` → `@Value` / `CafeneaProperties` → obiectul care îl folosește. Ai făcut primul capăt și ai oprit lanțul.

💡 **Hint:** ordinea cea mai ușoară: (1) `@Value("${cafenea.tva}")` pe parametrul din constructorul `CalculatorPret` + `@Component`, apoi șterge `@Bean`-ul; verifici cu `--cafenea.tva=0.19`. (2) `program.properties` + `@PropertySource`. (3) recordul `CafeneaProperties` și antetul.

🟡 Importante

M1 🟡 **`mvn test` pică.** `CafeneaSpringApplicationTests.java:6`: `Unable to find a @SpringBootConfiguration`. Testul generat de start.spring.io caută `@SpringBootApplication` în pachetul lui (`mycode`) și în părinții lui, iar tu ai mutat clasa în `mycode.pornire`, deci nu o mai găsește. Pe Windows, `mvn package` fără `-DskipTests` ar pica la fel. 💡 Gândește-te unde caută Spring (în sus, nu în jos) și ce ar face `CommandLineRunner`-ul tău dacă testul ar porni contextul (citește de la tastatură).

M2 🟡 **`@Import(AppConfig.class)` nu mai are rost.** `CafeneaSpringApplication.java:11`. Am șters rândul pe o copie și aplicația pornește la fel. Cu `@ComponentScan("mycode")` și `AppConfig` în `mycode.config`, scanarea îl găsește deja (`@Configuration` e un `@Component`). În cerință `@Import` era necesar fiindcă `config` stătea *în afara* pachetului `app` scanat. Redenumirea `app` → `mycode` a mutat configurația în interiorul zonei scanate și a dispărut motivul adnotării.

🟢 Cleanups

- **C1** `import ...stereotype.Component;` rămas nefolosit: `ProdusRepositoryImpl.java:4`, `ComandaRepositoryImpl.java:4`, `ProdusServiceImpl.java:5`, `ComandaServiceImpl.java:7`.
- **C2** `application.yml:13`: `program : 08:00–20:00` are spațiu înainte de `:`; YAML îl acceptă, dar scoate-l ca să arate la fel ca celelalte chei. (Oricum `program` se mută în `program.properties`, vezi B2.)
- **C3** `ViewCafenea` stă în `products.view`, dar lucrează și cu comenzile; un pachet `view` separat ar reflecta mai bine rolul ei.

Before / After

Înainte	După (indicativ, în document, nu în codul tău)
`@Configuration` `@PropertySource("classpath:application.yml")`	`@Configuration` `@PropertySource("classpath:program.properties")` (+ `src/main/resources/program.properties` cu `cafenea.program=08:00–20:00`)
`AppConfig`: `new CalculatorPret(0.21)`	`@Component` pe `CalculatorPret`; `@Value("${cafenea.tva}") double tva` în constructor; `@Bean`-ul dispare
`ViewCafenea`: `case 2:` buclă „Comanda noua”	`case 2 -> adaugaProdus(comanda)` ca în vanilla: o singură cerere de id, comanda rămâne aceeași

Q&A

1. De ce `application.yml` este citit fără nicio adnotare, dar `program.properties` are nevoie de `@PropertySource`? Ce citește fiecare fișier și cine?
2. Pe rândul 3 din tabelul de la B2, de ce lipsesc *amândouă* cheile, nu doar `cafenea.program`? Ce valoare ar avea cheia `nume` în acel `Properties`?
3. În cerința originală `@Import(AppConfig.class)` era necesar. Ce anume din structura pachetelor îl făcea necesar, și ce s-a schimbat când ai redenumit `app` în `mycode`?